Nesne Yönelimli/Tabanlı Programlama(OOP)

- Günümüzde nesne tabanlı programlama birçok modern dil tarafından desteklenir haldedir. Bunlara örnek olarak: C#, Python, JAVA, C++ vb. verilebilir.
- Nesne Yönelimli Programlama yazılım geliştirme için kullanılan bir yaklaşımdır(paradigmadır).
- Nesne Yönelimli Programlama yazılım geliştirme süreçlerini oldukça kısaltan ve sistematik hale getiren bir tekniktir.

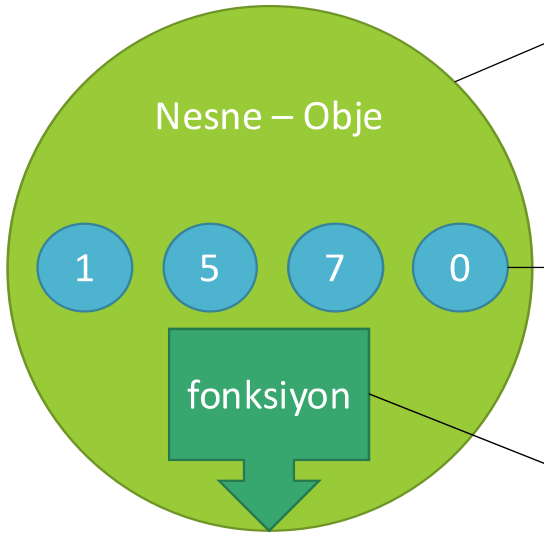
Nesne Yönelimli/Tabanlı Programlama(OOP)

- OOP gerçek hayatı, programlama için simüle eden nesneleri baz alır.
- Her şey bir nesnedir felsefesine dayanır.
    - Sen
    - Ben
    - Personel
    - Ürün
    - Satış
    - Araba
    - Kapı

Nesne Yönelimli/Tabanlı Programlama(OOP)

- Gerçek bir sistem, nesnel parçalara ayrılır ve bu parçalar(nesneler) arasında ilişkiler kurulur.
- Nesneler kendi aralarında haberleşebilirler.

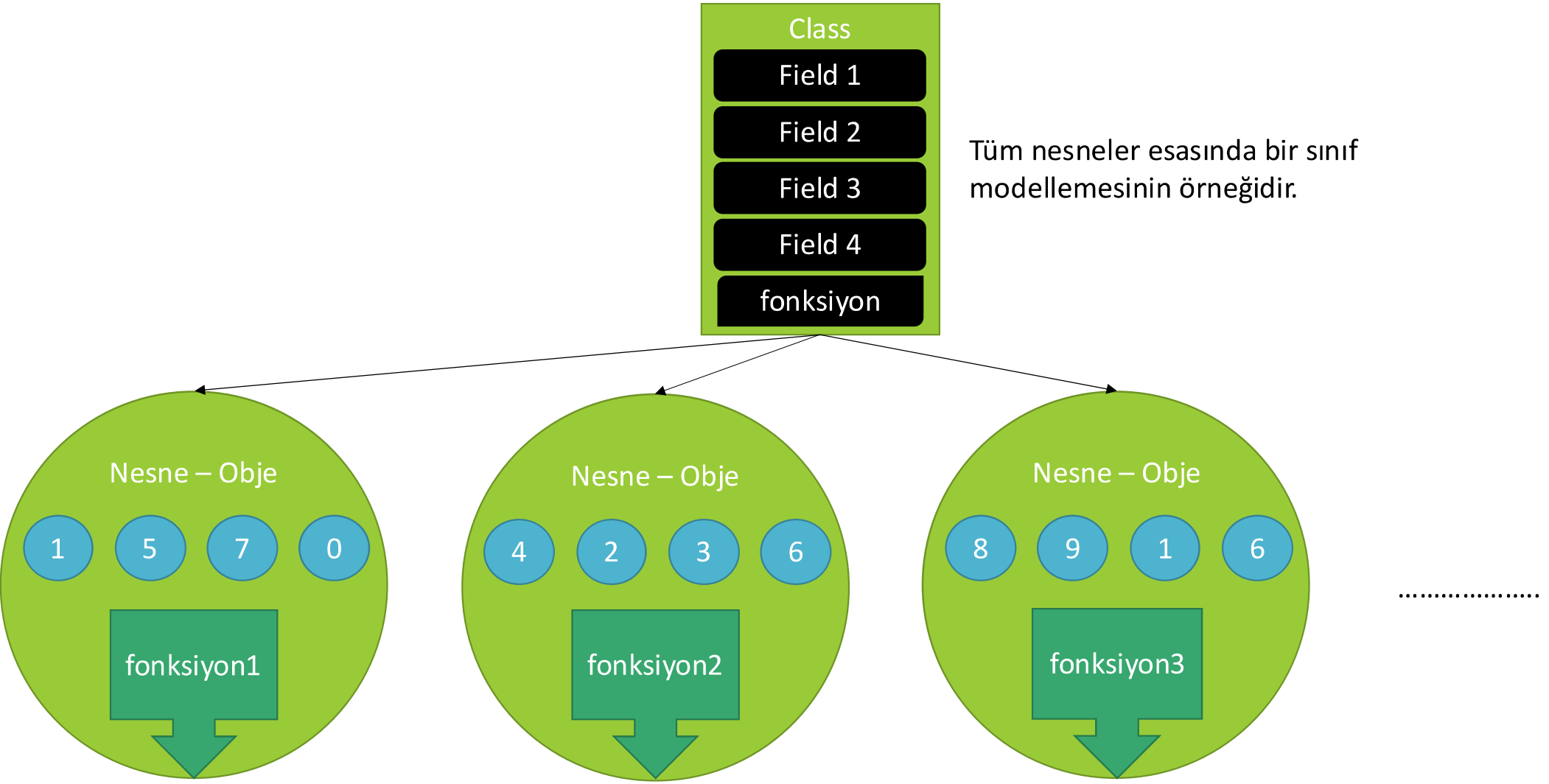
Nesnenin Anatomisi



Nesne Yönelimli Programlamada en küçük esas parça nesne – obje – object'tir.

Nesneler içerisinde veri tutabilecekleri alanlar barındırırlar. Biz bu alanlara field diyoruz

Nesneler içerisinde fieldlarda ki değerleri işleyebilmesi için fonksiyonlar mevcuttur. Bu fonksiyonları ileride metot, property veya indexer olarak kullanacağız.


Class
Field 1
Field 2
Field 3
Field 4
fonksiyon
Tüm nesneler esasında bir sınıf modellemesinin örneğidir.
Nesne – Obje
1
5
7
0
fonksiyon1
Nesne – Obje
4
2
3
6
fonksiyon2
Nesne – Obje
8
9
1
6
fonksiyon3
.....

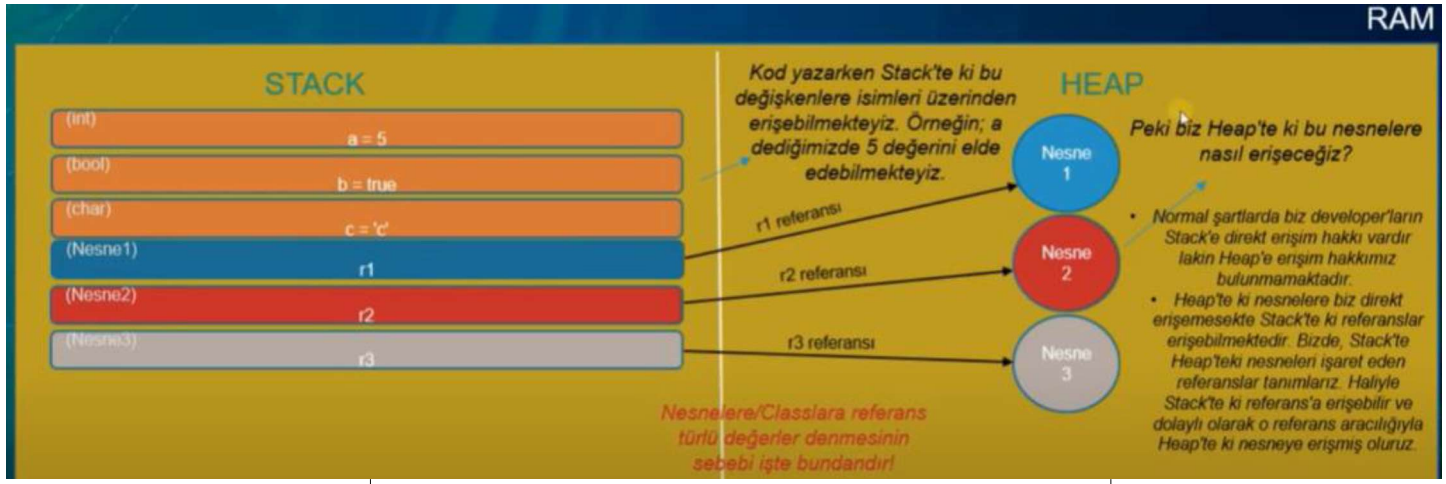
Nesne Kavramı

- Nesne, nesnellik felsefesine dayanan bir kavramdır. Kainattaki her şeyi bir nesne olarak görmek ve o şekilde yorumlamak fikrine dayanır.
- Nesne, gerçek hayatta elle tutulur, gözle görülür objelerdir. Dolayısıyla programlamada da nesnelerin bu şekilde olduğu düşünülebilir.
- Gerçek hayattaki herhangi bir olguyu, nesneyi, objeyi programlama dünyasında tarif ederken de onu bir nesne olarak tarif edecek ve o şekilde modelleyeceğiz.

Nesne Modellemesi

- Nesnelerin oluşturulabilmesi için öncelikle modellenmesi gerekmektedir.
- Nesne modeli class ile gerçekleştirilir.
- Class bir şablon nesne ise bu şablon kullanılarak üretilenler olarak düşünülebilir.
- Örneğin: öğrenci kimlik kartı, arabalar, uçaklar, kitaplar vb.

Referans Türlü Değerler



- Değer türlü değişkenler ve değerleri tutulur.
- Referansları tutulur.

- Sadece nesneler tutulur

Sorular

- Sınıf Nedir? Neden sınıf yapısı kullanılır?
- Sınıf ile Nesne arasındaki ilişki Nedir?
- Sınıf Nasıl ve Nerede Oluşturulur?
- Sınıf ile Nesne Modeli Tasarlama
- Sınıf Modelinden Referans Noktası Oluşturma

Örnek Class Tanımlaması

```
class Kapi
{

}
```

Class Üyeleri(Memberları)

- Class üyeleri temel olarak 4 başlık altında incelenir. Bunlar:

1. Field(Alan)
2. Property(Özellik)
3. Method(Metot)
4. İndexer(İndeksleyici)

Field(Alan)

- Nesne içerisinde veri depoladığımız/tuttuğumuz alanlardır.
- Class içerisindeki değişkenlerdir.
- Herhangi bir türden olabilir.
- Field’lar türüne özgü varsayılan değer alırlar.

Property(Özellik)

- Nesne içerisinde özellik/property sağlar.
- Property esasında özünde bir metottur. Yani programatik/algoritmik kodlarımızı inşa ettiğimiz bir metot olarak ifade edilir.
- Lakin fiziksel olarak metottan farkı parametre almaması ve içerisinde get ve set olmak üzere iki adet blok içermesidir.
- Property'nin değeri okunmak istediğinde get bloğu Property'e değer atanmak istediğinde de set bloğu çalışır.
- Property'nin işlevsel açıdan metottan farkı yoktur, ancak davranışsal olarak nesne üzerinden değer okuma ve atama için kullanılır.

Neden Property(Özellik) Kullanılır?

- Yazılımcılar nesneler içerisindeki fieldlara doğrudan erişilmesini istemez!
- Dolayısıyla field'lar da ki verileri kontrollü bir şekilde dışarı açmak isteriz. İşte böyle bir durumda metotları kullanabiliriz. C#'ta buna Property ile de izin verilmiştir.
- Yani Property yapıları özünde nesne içerisindeki bir fieldın dışarıya kontrollü açılmasını ve kontrollü bir şekilde dışarıdan değer almasını sağlayan yapılardır.
- İşte Property'lerin bu işlevine Encapsulation(Kapsülleme/Sarmalama) denir ve nesne yönelimli programlamanın ortak özelliklerindendir.

Encapsulation(Kapsülleme/Sarmalama)

- Encapsulation, bir nesne içerisindeki dataların(fieldlardaki verilerin) dışarıya kontrollü bir şekilde açılması ve kontrollü bir şekilde veri alınasıdır.

Property Tanımlama

[erişim belirleyicisi] [geri dönüş değeri] [property adı]
{
get {} // Property'den veri istendiğinde tetiklenir.
set{} /* Property'e veri gönderildiğinde tetiklenir. Gönderilen veriyi value keywordü yakalar.*/
}

- Eğer set bloğu tanımlanmaz ise sadece okunabilir(readonly), sadece get bloğu tanımlanmaz ise sadece yazılabilir(writeonly) property olacaktır.

Metot

- Nesne üzerinde, fieldlardaki yahut dışarıdan parametreler eşliğinde gelen değerler üzerinde işlemler yapmamızı sağlayan temel programatik parçalardır.

Indexer

- Nesneye indexer özelliği kazandıran onun dışında propety ile birebir aynı olan elemandır. Şu şekilde tanımlanır:

[erişim belirleyicisi] [geri dönüş değeri] this[parametreler]
{
get{} // Indexerdan veri istenildiğinde tetiklenir.
set{} /*Indexer'a veri gönderildiğinde tetiklenir. Gönderilen veriyi value keywordüyle yakalar*/
}

this Keywordü

- C#'ta this keywordü genel olarak 3 farklı amaç için kullanılır. Bunlar:

1. Sınıfın(Classın) nesnesini temsil etmek için kullanılır.
2. Aynı isimde field ile metot parametrelerini ayırmak için kullanılır.
3. Bir Constructor'dan(Yapıcı Metot) başka bir bir constructorı çağırmak için kullanılır.

Not: this keywordü class memberları(üyeleri) içerisinde kullanılır!

this Keywordü

```
class Ogrenci
{
    public int vize_notu { get; set; }
    public string isim { get; set; }

    public void HarfNotuHesapla(string isim, int vize_notu)
    {
        isim = "Zafer";
        this.vize_notu = 100;
    }
}
```

this Keywordü

- this keywordü ilgili class yapılanmasının o anki nesnesine karşılık geliyorsa önceden bahsedilen 3 durum dışında da this kullanmak gerekmez mi?
- C#'ta bahsedilen bu 3 durum dışında this keywordünü kullanma zorunluluğu yoktur. Bununla birlikte compiler(derleyici) seviyesinde yine this kullanılmaktadır. Python'da(self keywordü) veya typescriptte(this keywordü) bu keyword mutlaka yazılmalıdır!

Constructor(Yapıcı Metot)

- Constructor bir nesne üretimi sürecinde ilk tetiklenen(çalışan) metottur.
- Constructor, nesne oluşturma sürecinde tetiklenmek zorundadır!
- Nesne ilk oluşturulduğunda yapılmak istenen işlemler Constructor içerisinde belirtilir. Zorunlu olmamakla birlikte genel olarak class fiedlarına ve propertylerine başlangıç değerleri atamak için kullanılır. Nesne ile ilgili başlangıç konfigürasyonlarını(ayarlamalarını) yapmak için kullanılır.
- Constructor, new ile nesne oluşturma talebi geldikten ve ilgili nesneye hafızada yer ayrıldıktan sonra tetiklenir.

Constructor(Yapıcı Metot) Nasıl Oluşturulur?

- Constructor özel bir sınıf elemanıdır; ancak temelde bir metottur.
- Bir metot olmasına karşın imzası bir nebze farklıdır ve Constructorlar şu özelliklere sahiptir:

1. Constructor adı sınıfın adıyla aynı olmalıdır! (Özel sınıf elemanlarının dışında hiçbir class memberı(üyesi) sınıf adıyla aynı adda olamaz!)
2. Geri dönüş değeri olmaz/belirtilmez!
3. Erişim belirleyicisi public olmalıdır!(private olduğu durumu ayrıca inceleyeceğiz.)

Constructor(Yapıcı Metot) Nasıl Oluşturulur?

```
class Ogrenci
{
    public Ogrenci()
    {
        // Constructor metot içerisinde yapılacaklar
    }
    public void X()
    {
        // Normal metot içerisinde yapılacaklar. Bu constructor değildir!
    }
}
```

Default Constructor

- Her sınıfın içerisinde tanımlamasak dahi default bir constructor mevcuttur.
- Şimdiye kadar yaptığımızı örneklerde bir nesne oluştururken şu kodu yazdık: ‘Ogrenci ogrenci1 = new Ogrenci();’ buradaki () biz oluşturmasak dahi compiler(derleyici) seviyesinde varsayılan olarak bir constructor oluşturulduğunu belirtir.
- Eğer ki bir classa constructor eklersek bu default constructorı ezmiş(override) oluruz.

Parametreli Constructor ve Constructor Overloadı

- Constructorlar parametre alabilmektedirler.
- Constructorlar overload(aşırı yükleme) işlemine tabi tutulabilirler. Overload aynı isimde ki bir metodun farklı sayıda veya tipte parametreler ile çağrılarak birbirinden ayrılması ama aynı isimle çağrılmasıdır.

Private Constructor

- Bir constructorın erişim belirleyicisi private yapılırsa o class dışından nesne üretimi engellenmiş olur!
- Bu durum Singleton design pattern(tasarım deseni) yapısında çok sık kullanılır. Bu durumda ilgili constructor dışarıdan erişilemez lakin classın içerisinden erişilebileceği için ilgili işlemler burada yapılabilir.

this keywordüyle Constructorlar Arası Geçiş

```
class Ogrenci
{
    public Ogrenci() : this(42)
    {
        Console.WriteLine("Parametresiz yapıcı metot.");
    }
    public Ogrenci(int a)
    {
        Console.WriteLine("Tek parametreli yapıcı metot.");
    }
    public Ogrenci(int a, int b) : this()
    {
        Console.WriteLine("2 parametreli yapıcı metot.");
    }

}
```

Destructor(Yıkıcı Metot)

- Bir classtan üretilmiş olan nesne imha edilirken otomatik çağrılan metottur.
- Destructor özellikleri

1. Destructor sadece class yapılanmasında kullanılabilir.
2. Bir classta sade ve sadece bir Destructor bulunabilir.
3. Destructor parametre alamaz, erişim belirteci olmaz.
4. Destructor overload işlemlerine tabi tutulamaz.
5. Destructor class ile aynı isimde olmalı ancak başında ~(tilda) işareti olmalıdır.

Nesne Hangi Şartlarda Kim Tarafından İmha Edilir?

- Bir nesnenin imha edilmesi için;

1. İlgili nesne herhangi bir referans tarafından işaretlenmemelidir.
2. Nesnenin oluşturulduğu ve kullanıldığı scope(faaliyet alanı) sona ermiş olmalıdır.
3. Yani ilgili nesne bir daha erişilemez hale gelmelidir.

- Bu durumlarda nesne Garbage Collector(Çöp Toplayıcısı) tarafından imha edilir.

Garbage Collector(Çöp Toplayıcısı)

- Uygulamada lüzumsuz olan nesneleri toplamak için Garbage Collector isimli bir mekanizma devreye girer.
- Esasında Garbage Collector c#'ta bellek optimizasyonunu üstlenen bir yapılanmadır.
- C#'ta Garbage Collector'ın ne zaman iş göreceği tahmin edilemez.
- Dolayısıyla geliştiricilerin bu mekanizmaya müdahale etmesi pek önerilmez.

Destructor Tanımlama Kuralları

```
class Ogrenci
{
    ~Ogrenci()
    {
        // işlemler
    }
}
```

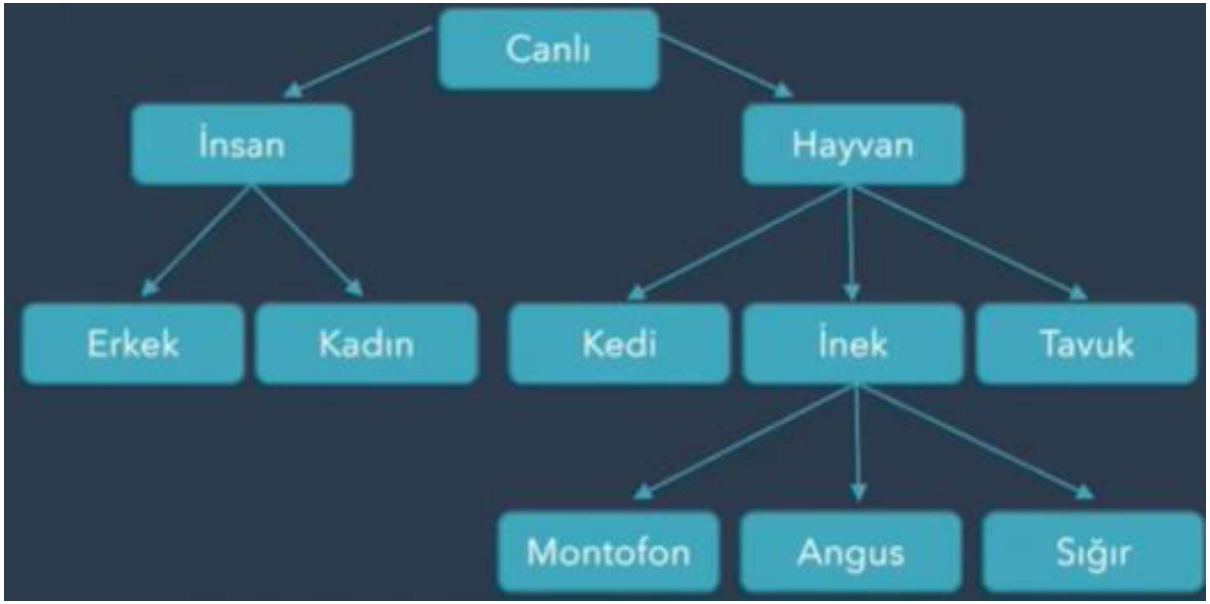
static Constructor

- static anahtar sözcüğü kullanıldığı yapıyı nesne üretilmeden, doğrudan sınıf üzerinden de erişilebilir yapmak için kullanılır.
- Örneğin bir metot static olarak işaretlenirse bu metodun tanımlandığı classtan bir nesne üretilmese dahi bu metoda class adı ile erişilebilir. Aynı durum class memberlarının tamamı için geçerlidir.
- Static Constructor bir classtan nesne üretildiğinde normal constructordan bile önce tetiklenen bir metottur. Normal constructordan farkı ise ilgili classtan 50 tane nesne oluşturulursa normal constructor 50 kez tetiklenirken static constructor yalnızca 1 kez tetiklenir.

Kalıtım(Miras Alma - Inheritance)

- Kalıtım OOP'nin en önemli özelliğidir.
- Üretilen nesneler farklı nesnelere özelliklerini aktarabilmekte ve böylece hiyerarşik bir düzenleme yapılabilmektedir.
- Bir programcı açısından bu özellik;
  - Aynı aile grubundan gelen nesnelerin ya da yatayda eşit seviyede olan tüm olguların benzer özelliklerini tekrar tekrar her birinde tanımlamaktansa bir üst sınıfta tanımlanmasını ve her bir sınıfın bu özellikleri üst sınıftan kalıtımsal olarak almasını sağlamaktadır.
  - Böylece kod maliyeti düşer ve mimarisel tasarım avantajı sağlanır.

Kalıtım(Miras Alma - Inheritance)

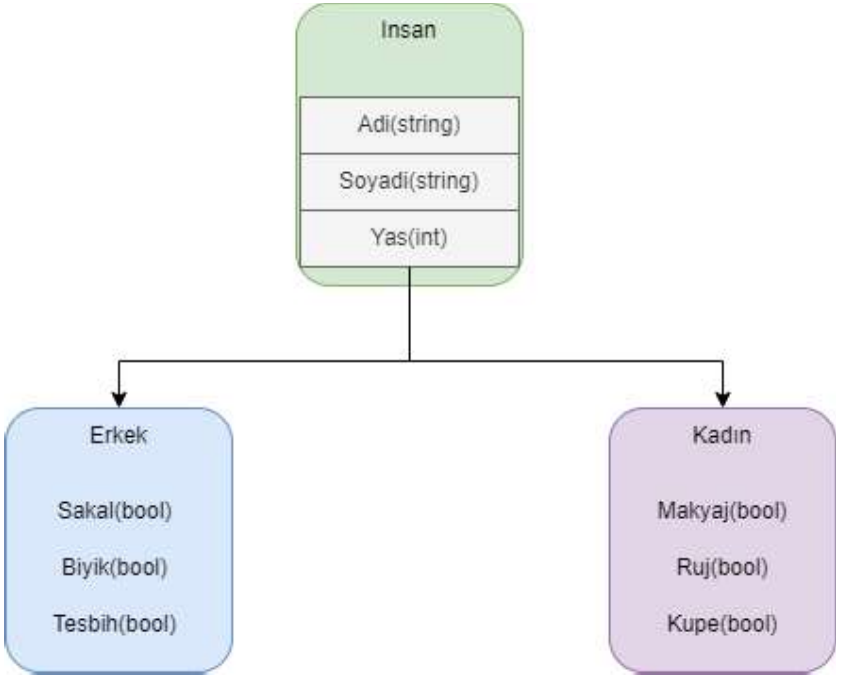


Kalıtım(Miras Alma - Inheritance)



- Nesne tabanlı programlamada, benzer/aynı olgudaki nesnelerin aynı olan memberları/özellikleri/içerikleri eğer ki bu şekilde her sınıf içinde tekrar tekrar tanımlanmışsa bu aykırı bir durumdur!
- Aynı olguda olan sınıfların tekrar eden memberları başka bir sınıfta tanımlansın ve kalıtım ile diğer sınıflara aktarılsın.

Kalıtım(Miras Alma - Inheritance)



- Erkek ve Kadın sınıfları Insan sınıfından kalıtım alırsa/türetilirse/miras alırsa insan sınıfındaki tüm memberlar(erişimine izin verilen/miras olarak aktarılmasına izin verilen memberlar) miras olarak aktarılacaktır.
- Genellenemeyen, diğerlerinde olmayan ve sadece o sınıfa ait olan özellikler direkt ilgili sınıfta tanımlanmalıdır.

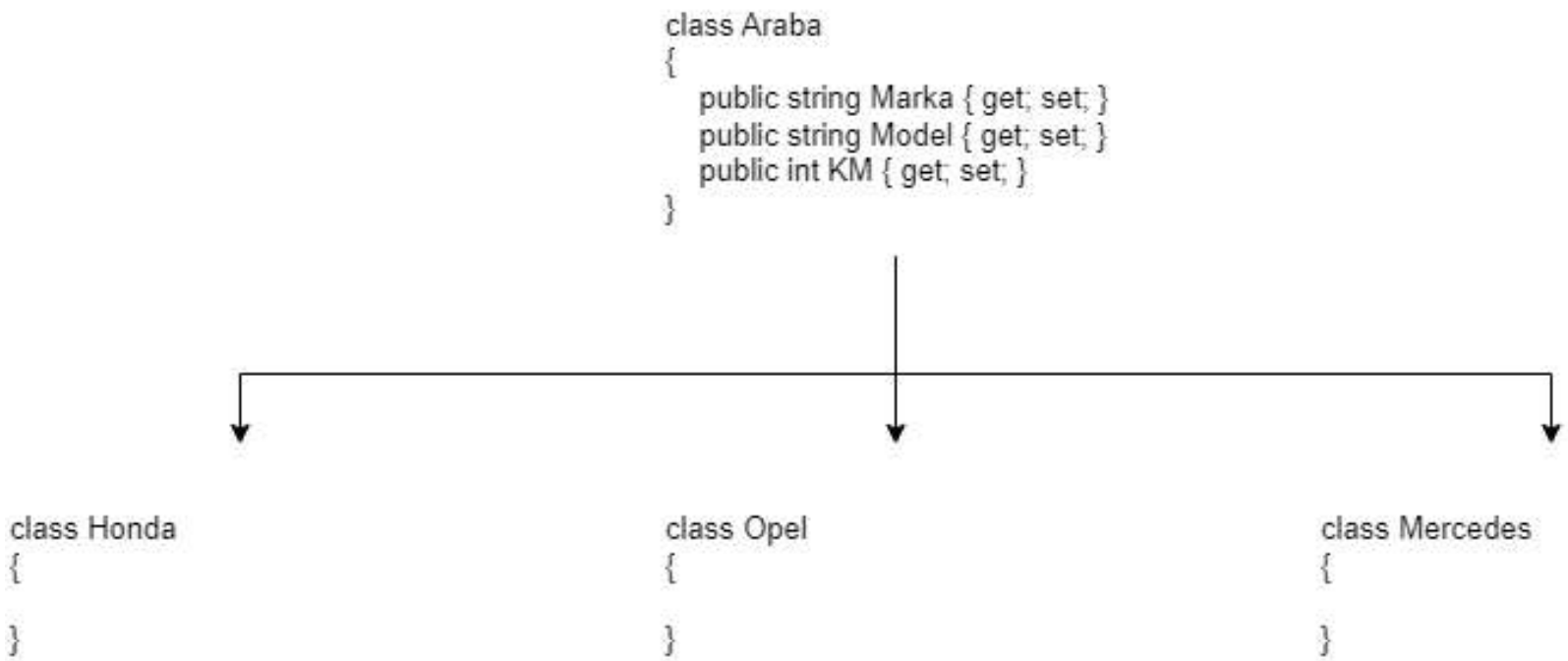
Kalıtım(Miras Alma - Inheritance)

```
class Honda
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int KM { get; set; }
}
```

```
class Opel
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int KM { get; set; }
}
```

```
class Mercedes
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int KM { get; set; }
}
```

Kalıtım(Miras Alma - Inheritance)



Kalıtım(Miras Alma - Inheritance)

- Kalıtım rastgele bir şekilde tasarlanmamalıdır! Ortak olguda olan nesneleri temsil edecek olan bir üst ve daha evrensel nitelikte olgu olmalıdır. Honda, Opel ve Mercedes ortak olgudur. Yani üçünde bir arabadır. Haliyle bunların daha evrensel üst niteliği Araba olarak nitelendirilebilir.
- Kalıtım operasyonunda, kalıtım veren sınıfın erişilebilen tüm memberları kalıtım alan sınıfa kalıtsal olarak aktarılacaktır.

Kalıtım(Miras Alma - Inheritance)

- OOP'de kalıtım özünde nesnelerin birbirlerinden türemesini sağlayan bir özelliktir.
- Bu özellik yanında da birçok özellik ve stratejik yapılanma getirmektedir.
- C#'ta iki sınıf arasında kalıtımsal ilişki kurabilmek için : operatörü kullanılmaktadır.

Kalıtım(Miras Alma - Inheritance)

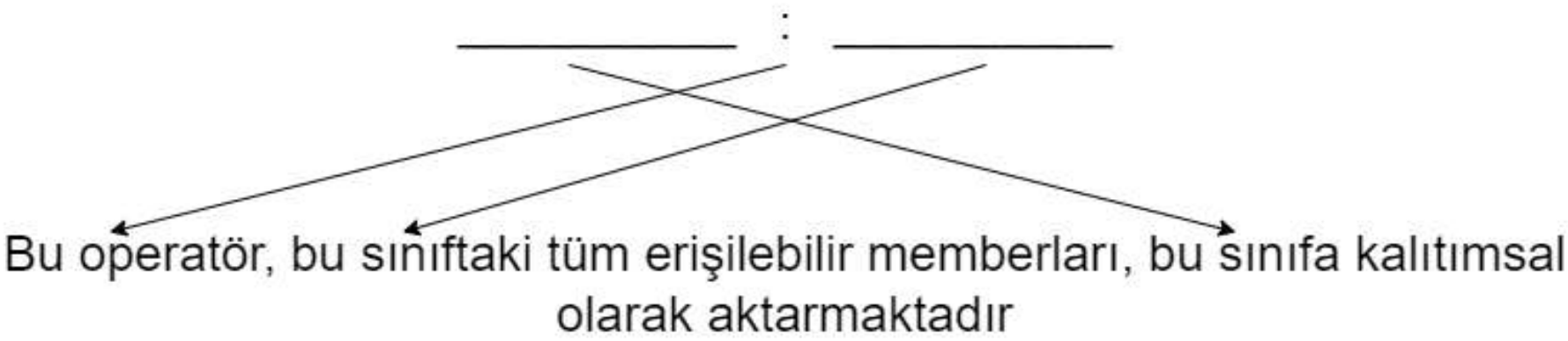
```
class Araba
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int KM { get; set; }
}
```

```
class Honda : Araba
{

}
```

- : operatörünün solundaki class sağındaki classtan kalıtım yoluyla erişime açık olan memberları alacaktır.
- Yani; Honda sınıfı, Araba sınıfından kalıtım alsın demiş oluyoruz.

Kalıtım(Miras Alma - Inheritance)



Kalıtım(Miras Alma - Inheritance)

- Kalıtım, operasyonel olarak gerçekleştirildikten sonra compiler seviyesinde member aktarımı sağlanır.
- Yani artık Honda sınıfından bir nesne ürettiğimizde içerisinde Marka, Model ve KM propertyleri kalıtımsal olarak aktarıldığı için erişilebilir olacaktır.

Kalıtım(Miras Alma - Inheritance)

```
class Araba
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int KM { get; set; }
}
```

```
class Honda : Araba
{

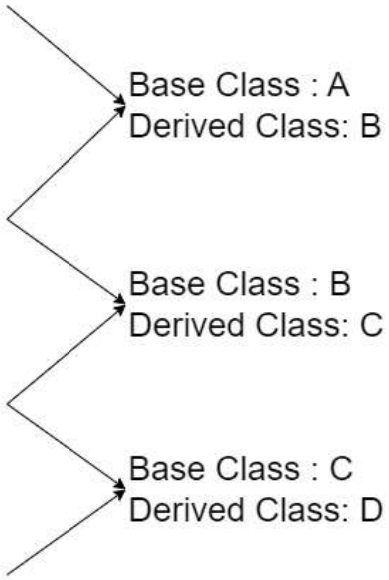
}
```

- Kalıtım veren sınıfa Base / Parent Class denir.
- Kalıtım alan sınıfa Derived / Child Class denir.
- Bu örnek için Base Class Araba iken Derived Class Honda'dır.

Kalıtım(Miras Alma - Inheritance)

```
class A
{
...
}

class B : A
{
...
}
class C : B
{
...
}
class D : C
{
...
}
```

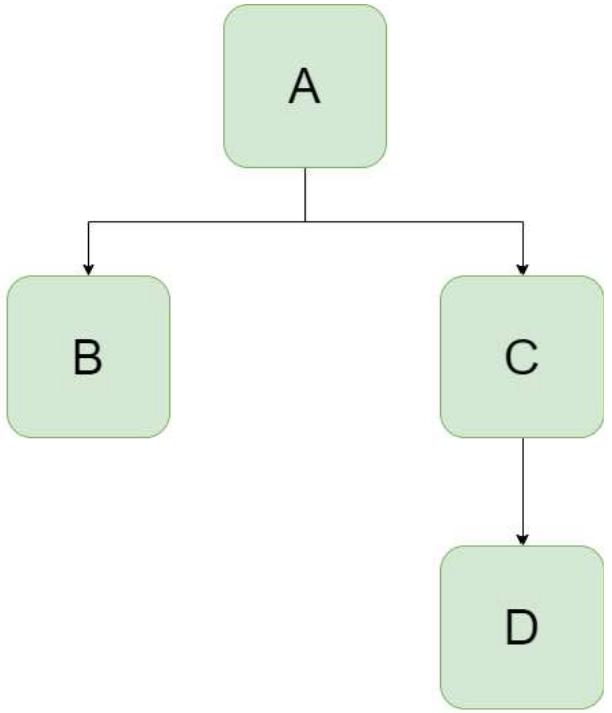


- Peki, atalar tüm torunların Base Class'ı mıdır? Yani A, B'nin olduğu gibi bir yandan da C ve D'nin de Base Class'ı mıdır?
- Hayır değildir!
- Bir sınıfın sade ve sadece tek bir Base Class'ı olabilir.
- Yani bir sınıfın Base Class'ı direkt türediği sınıftır.
- Lakin atalarındaki tüm classlar Base Class'ı değildir.
- Örneğin; C'nin Base Class'ı B'dir. A ise atasıdır lakin Base Class'ı değildir.
- Buna karşın bir Class'ın birden fazla derived classı olabilir.

Kalıtım(Miras Alma - Inheritance)

```
class A
{
...
}

class B : A
{
...
}
class C : A
{
...
}
class D : C
{
...
}
```



Kalıtım(Miras Alma - Inheritance)

- Kalıtımın Altın Kuralı: Bir classın sade ve sadece bir Base Class'ı olabilir. Dolayısıyla bir classı hem A classından hem de B classından yatayda(aynı anda) türetmek mümkün değildir! Dikeyde bu işlem yapılabilir. Örneğin A classını aynı anda hem B'den hem de C'den türetemezsiniz, ancak B classını C'den türetip A classınıda B'den türetebilirsiniz.
- Bunun nedeni, c# programlama dilinde bir classın sade ve sadece tek bir classtan türetilmesine izin verilmesidir! Aynı anda birden fazla classtan türeme işlemi gerçekleştirilemez!

Kalıtım(Miras Alma - Inheritance)

- Bu durumda aşağıdaki gibi bir kullanım olamaz! (Not: bu durum classlar için geçerlidir. İleri düzey programlama yapılarından olan interface buna izin verecektir; ancak interface bir class değildir!)

```
class Y : X, Z, W
{

}
```

Kalıtım(Miras Alma - Inheritance)

- Bir sınıftan nesne üretimi yapılırken kalıtım aldığı üst sınıflar varsa eğer önce o sınıflardan sırayla nesne üretilir. Örneğin new B() kodu ile aşağıdaki kod çalıştığında Heap'te önce A'dan sonra B'den nesne üretilir.

```
class A
{
}
class B : A
{
}
```

Kalıtım(Miras Alma - Inheritance)

- Yani bir sınıftan nesne üretilirken 1 adet nesne üretildiği düşünülse de kalıtımsal açıdan birden fazla nesne üretimi gerçekleştirilebilmektedir.

Kalıtım(Miras Alma - Inheritance)

- Herhangi bir sınıftan nesne üretimi gerçekleştirilirken öncelikle base classından nesne oluşturuluyor ise bu demektir ki önce base classın constructorı tetikleniyor.
- Haliyle bizler nesne üretimi esnasında base classta üretilecek olan nesnenin istediğimiz constructorlarını tetikleyebilmeli veya varsa parametrelerini verebilmeliyiz.
- İşte bu amaçlar için base keywordü kullanılır.

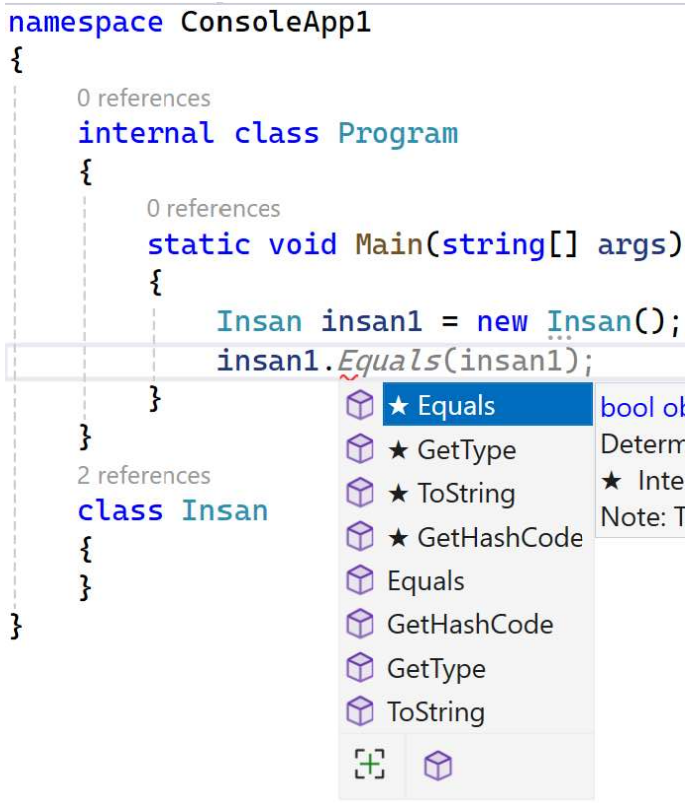
Kalıtım(Miras Alma - Inheritance)

- Eğer ki base classın constructorı sadece parametre alan constructor ise derived classlarda o constructora bir değer göndermek zorundayız! Bunu da base keywordü ile sağlayabiliriz.
- Eğer ki base classta boş parametreli bir constructor varsa derived classta base ile bir bildirimde bulunmak zorunda değiliz. Çünkü varsayılan olarak kalıtımsal durumda boş parametreli constructor tetiklenir.
- Bir classın constructorının yanında :base() keywordü kullanılırsa; o classın base classının tüm constructorları bize getirilecektir. Bu sayede ilgili sınıftan bir nesne üretilirken base classın hangi constructorı tetiklenecek belirtebiliriz.

Kalıtım(Miras Alma - Inheritance)

- this, bir sınıftaki constructorlar arasında geçiş yapmamızı sağlar.
- base, bir sınıfın base classının constructorlarından hangisinin tetikleneceğini belirlememizi ve varsa parametrelerinin değerlerinin derived classtan verilmesini sağlar.
- Buna ek olarak nasıl ki this, ilgili sınıfta o anki nesnenin memberlarına erişebilmemizi sağlıyor, aynı şekilde basede base classtaki memberlara erişebilmemizi sağlar.

Kalıtım(Miras Alma - Inheritance)



- Nesnelerdeki ToString(), Equals(), GetHashCode() ve GetType() gibi metotlar c#'ta bütün nesnelerin atası olan Object sınıfından gelmektedir.

Kalıtım(Miras Alma - Inheritance)

- C# programlama dilinde tüm sınıflar Object sınıfından türetilir. Bu işlem compiler seviyesinde gerçekleştirilir. Dolayısıyla bazı metotlar bu sayede alt sınıflara(torunlara) aktarılabilecektir.
- Dolayısıyla önceden gördüğümüz object yapısı Object classına ait bir yapı olduğu ve c#'taki tüm classlardan Object classından türetildiği için object anahtar sözcüğü ile herbir yapı tutulabilir, boxing ve unboxing işlemine tabi tutulabilir.

Kalıtım(Miras Alma - Inheritance)

- Eğer ki tanımlanan sınıf herhangi bir kalıtım almıyorsa default(varsayılan) olarak Object sınıfından türetilecektir.
- Eğer ki tanımlanan sınıf herhangi bir sınıftan kalıtım alıyorsa, bir sınıfın aynı anda birden fazla sınıftan kalıtım almama prensibinden yola çıkarak bir yandan da Object sınıfından türemeyecek sadece kalıtım aldığı sınıftan türeyecektir.
- Tabi burada kalıtım veren sınıf herhangi bir sınıftan türemiyorsa eğer en nihayetinde Object'ten türeyeceği için dolaylı yoldan en alt sınıfta Object'ten kalıtım almış olacaktır.

Erişim Belirteçleri

Erişim Belirteci	İşlevi
public	Erişim kısıtı yoktur, her yerden erişilir.
protected	Ait olduğu sınıftan ve o sınıftan türetilen sınıflardan erişilebilir.
internal	Etkin projeye ait sınıflardan erişilebilir, onların dışından erişilemez.
private	Yalnız bulunduğu sınıftan erişilir, dıştaki sınıflardan erişilemez.
protected internal	Etkin projeye ait sınıflardan ve onların türevlerinden erişilebilir.
private protected	Yalnız bulunduğu sınıftan ve onların türevlerinden erişilebilir

Sanal Yapılar Virtual & Override

- Bir nesne üzerinde var olan tüm memberlar(üyeler) derleme zamanında belirgindir.
- Yani, derleme aşamasında hangi nesne üzerinden hangi metotların çağrılabileceği bilinmektedir.

```
class MyClass
{
    0 references
    public int MyProperty { get; set; }
    0 references
    public void MyMethod() { }
}
```

- Derleme zamanından olaya bakıldığında MyClass sınıfından üretilen tüm nesnelerde MyPproperty ve MyMethod çağrılabilecektir.

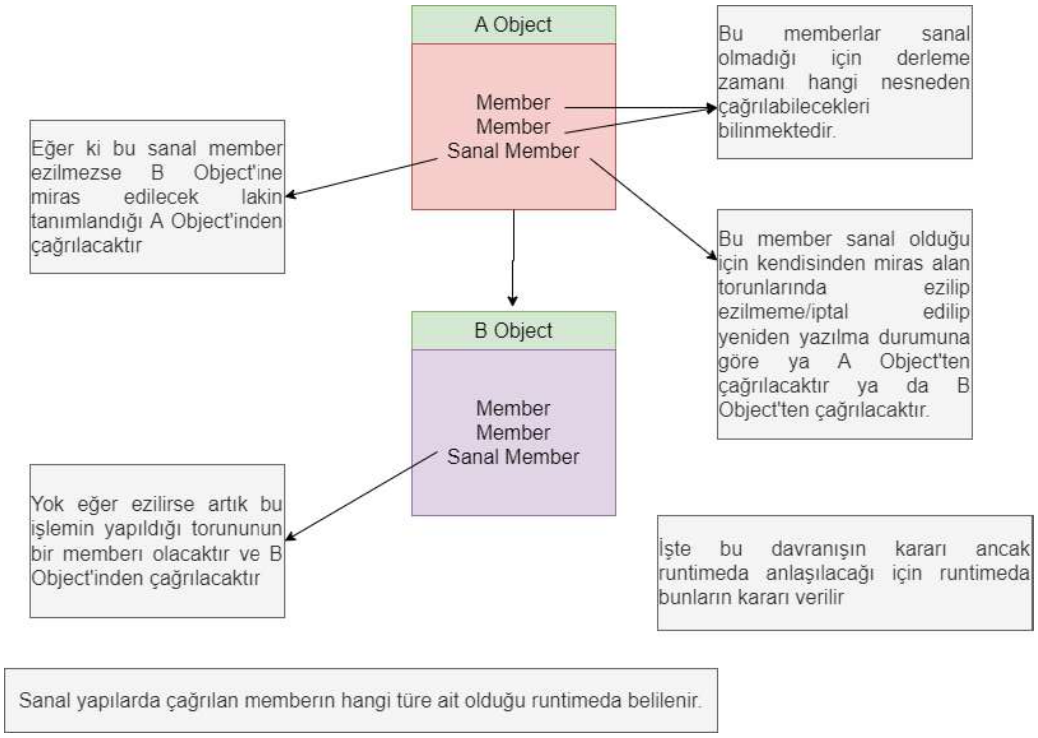
Sanal Yapılar Virtual & Override

- Sanal yapılar ile derleme zamanında kesin olarak bilinen bu bilgi runtime(çalışma zamanı) için de belirlenebilmektedir. Yani ilgili nesnenin hangi metodu kullanacağı runtimeda kararlaştırılır.
- Sanal yapılar, bir sınıf içerisinde bildirilmiş olan ve o sınıftan türeyen alt sınıflarda da tekrar bildirilebilen yapılardır.
- Bu yapılar metot, property veya indexer olabilir.
- Sanal yapılanmalarda üstten gelen bir yapının işlevi iptal edilip yeniden yapılandırılır.

Sanal Yapılar Virtual & Override

- Bir sınıfta tasarlanmış sanal yapının işlevinin iptal edilip edilmeme durumuna göre tanımlandığı sınıftan mı yoksa bu sınıfın torunlarından mı çağrılacağının belirlenmesi runtime da gerçekleşecektir.

Sanal Yapılar Virtual & Override



Sanal Yapılar Ne için Kullanılır?

- Bir sınıfta tanımlanmış olan herhangi bir memberın kendisinden türeyen alt sınıflarda ezilmesi için kullanılır.
- Peki bu zorunlu mudur? Yani bir sanal yapı illaki kendisinden türeyen torunlarda ezilmek zorunda mıdır?
- Tabiki değildir. Yani bir member sanal yapıldı diye illa ki kendisinden türeyen alt sınıflarda ezilmek zorunda değildir! Ama ezilmek istenirse direkt ilgili sınıfın bir memberı olacak şekilde çalışılmasını sağlamış olur.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir sınıfta sanal yapılar oluşturabilmek için ilgili memberın imzasını **virtual** keywordü ile işaretlemek yeterlidir.
- public virtual ya da virtual public şeklinde kullanılabilir.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir sınıfta sanal yapılar oluşturabilmek için ilgili memberın imzasını **virtual** keywordü ile işaretlemek yeterlidir.
- public virtual ya da virtual public şeklinde kullanılabilir.

```
class MyClass
{
    // Normal metot ve property tanımlama
    0 references
    public int MyProperty { get; set; }
    0 references
    public void MyMethod() { }
}
```

```
class MyClass
{
    // Sanal metot ve sanal property tanımlama
    0 references
    public virtual int MyProperty { get; set; }
    0 references
    virtual public void MyMethod() { }
}
```

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir class'ta virtual ile işaretlenerek sanal hale getirilmiş bir member, bu classtan miras alan torunlarında ezilmek isteniyorsa eğer ilgili classta imzası **override** keywordü ile işaretlenmiş bir vaziyette tekrardan aynı member oluşturulur.

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

```
class MyClass
{
    1 reference
    virtual public void MyMethod() { }
}

0 references
class MyClass2 : MyClass
{
    1 reference
    public override void MyMethod()
    {
    }
}
```

- Base classta virtual ile işaretlenen member ezilmek zorunda değil; ancak bir torun class base classtaki herhangi bir memberı ezecekse o member mutlaka virtual ile işaretlenmiş olmalı!

Bir Sınıfta Sanal Yapı Nasıl Oluşturulur?

- Bir classta virtual ile işaretlenmiş herhangi bir member illa ki direkt o classtan türeyen 1. dereceden classlarda override edilmek zorunda değil! İhtiyaca göre alt seviyedeki torunlardan herhangi birinde override edilebilir.
- Ancak böyle bir durumda virtual member en son override edildiği Object'ten sonra geçerli olur.

Çok Biçimlilik(Polimorfizm) Nedir?

- Polimorfizm esasında kalıtım gibi bir biyolojik terimdir.
- Polimorfizm biyolojik açıdan iki veya daha fazla farklı fenotipin aynı tür popülasyonunda bulunmasıdır.
- Polimorfizm yazılımda iki veya farklı nesnenin aynı tür classlardan(referanslardan) referans almasıdır.(işaretlenebilmesidir).
- Bir diğer tanımda Polimorfizm bir nesneyi birden fazla form ile tarif edebilmektir.

Çok Biçimlilik(Polimorfizm) Nedir?

- OOP'de polimorfizm, iki ya da daha fazla nesnenin aynı tür sınıf tarafından karşılanabilmesi/referans edilebilmesidir.
- Bir başka deyişle, bir nesnenin birden fazla farklı türdeki referans tarafından işaretlenebilmesi polimorfizmdir.
- Bir başka tanım Polimorfizm, bir nesnenin kalıtımsal olarak ilişkisi olan diğer nesnelerin referanslarıyla işaretlenebilmesini sağlar.

Çok Biçimlilik(Polimorfizm) Nedir?

- Polimorfizm, OOP tasarımlarında geliştirilen koda daha manevrasal bir şekilde nitelik kazandıran ve yaklaşım sergilememizi sağlayan bir özelliktir.
- Polimorfizm, programlamada ki temel prensiplerden olan ‘Sol/Sağ’ prensibini aşıp, eldeki nesnenin birden fazla referans ile işaretlenebilmesini sağlar.

Çok Biçimlilik(Polimorfizm) Nedir?

- Bir nesnenin birden fazla referansla işaretlenmesi; o nesnenin, birden fazla türün davranışını gösterebilmesini sağlar.

```
class Test{}
0 references
internal class Program
{
    0 references
    static void Main(string[] args)
    {
        Test t1 = new Test();
    }
}
```

Çok Biçimlilik(Polimorfizm) Nedir?

- Kuş deyince aklımıza kartal, papağan, deve kuşu, tavus kuşu vb. gelebilmektedir. Bunların hepsi temelde bir kuştur. Bununla birlikte kartal hem kuş özelliklerine hem de kartal özelliklerine sahip yani çok biçimlidir.
- Üstteki maddeye göre ortak atadan gelen, kalıtımsal olarak ‘Kuş’ tan türeyen tüm hayvanlar kendi türleri yahut ‘Kuş’ türü ile referans edilebilirler.
- Buradan da şunu anlıyoruz ki, yazılımsal açıdan çok biçimliliğin söz konusu olabilmesi için teknik olarak Kalıtım olması gerekmektedir!

Programlama da Polimorfizm Nerede Kullanılır?

- Bilinçsiz tür dönüşümü,
- Object türünün herhangi türdeki bir değeri alabilmesi,

Soyutlama(Abstraction)

- Soyutlama bir mantıktır, bir davranıştır.
- Tanım olarak soyutlama bir iş yaparken sadece ihtiyaç duyulan yapıların bulunması durumudur.
- Programlama açısından kullanıcı işlemleri için kullanılacak bir classta yüzlerce member olabilir, buna karşın sadece kullanıcı kayıt etmek isteyen bir kişiye bu metodun gösterilmesi yeterli olacaktır. İşte bu soyutlamadır.
- Bir başka deyişle soyutlama gerekli olanları göster, gereksiz olanları gösterme demektir.

Soyutlama(Abstraction)

- Soyutlama ile bir sınıfın memberlarından ihtiyaç noktasında alakalı olanları gösterip, alakalı olmayanları göstermemektir.

Abstract Class

- Abstract Class kullanılarak abstraction uygulanabilir.
- Abstraction = abstract class şeklinde bir düşünce yanlıştır. Bunlar aynı kelime kökenine sahip olmasına karşın abstraction interface ile de uygulanabilir. Burada dikkat edilmesi gereken abstraction davranışının abstract class ile uygulanabilecek olmasıdır.
- Bir sınıfın uyması gereken temel yapıyı tanımlamak için abstract class yapısı kullanılabilir ve gerekli modellemeyi gerçekleştirebilirsiniz.

Abstract Class

- Abstract classlardan normal şartlarda nesne üretilemez!

Interface(Arayüz) Nedir?

- Programlama süreçlerinde Interface yapılanması, nesnelere direkt olarak bir arayüz/şablon oluşturulmasını ve bu arayüz üzerinden geliştirici ile nesne arasındaki etkileşimin daha da kolaylaştırılmasını sağlayan bir araçtır.
- Hatta sadece geliştirici ile nesne arasındaki süreci kolaylaştırmamakta, ayrıca bir programın farklı bir programla veya bileşenle etkileşimini de kolaylaştırmaktadır.

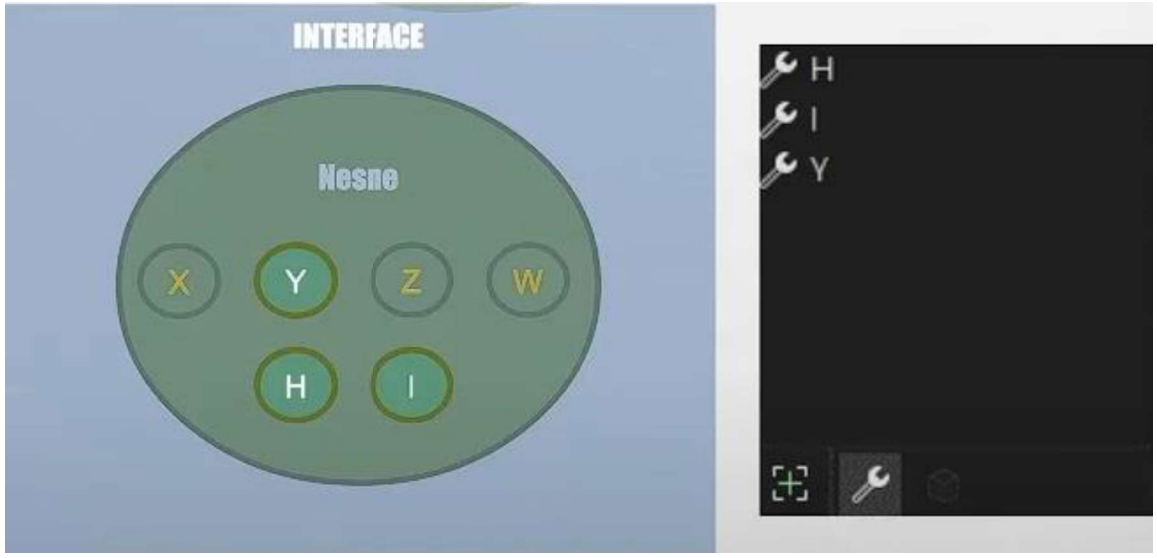
Interface(Arayüz) Nedir?

- Interface'in nesneye bir arayüz sağlaması, kullanıcı açısından ilgili nesnenin nasıl çalıştığına dair ayrıntılı bilgiye ihtiyaç duyulmaksızın, sadece arayüzün sunduğu fonksiyonların veya propertyleri kullanarak etkileşime girilmesini sağlar.
- Interface ilgili nesneye abstraction(soyutlama) uygulayarak belirlenmiş interface üzerinden çalışılmasını ve böylece ilgili nesne ile geliştirme sürecinin kolaylaştırılmasını sağlamaktadır.

Interface(Arayüz) Nedir?

- Abstract classlarda bir nesnenin içerisindeki imzaları modellemenin dışında farklı işlemlerde gerçekleştirilebilirken interfaceler sadece imzalara odaklanan bir yapıdır denilebilir.
- **Interface, abstract classın sadece imzalara konsantre olmuş halidir! O yüzden abstraction(soyutlama) davranışı açısından abstract classa nazaran daha elverişlidir!**

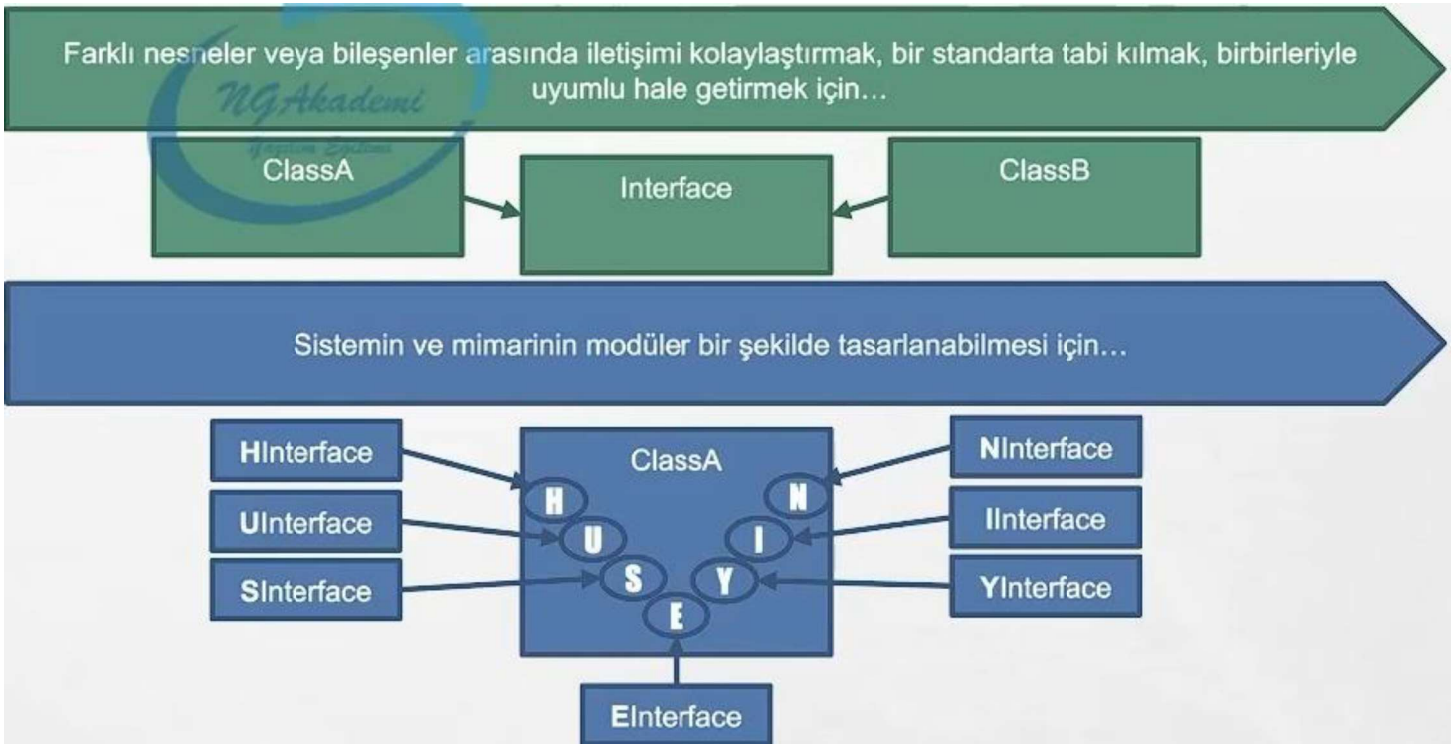
Interface(Arayüz) Nedir?



Interface(Arayüz) Nedir?

- Bizler bir sınıfta(classta) olmasını istediğimiz memberları oluşturmadan önce bir interfacete bu memberların imzalarını tanımlayarak bir şablon oluşturuyoruz.
- Ardından bu interface i kullanarak sınıfta memberları tanımlıyoruz.
- Bir sınıf, bir interface kullanıyorsa eğer aynen abstract classlardaki abstract elemanlarda olduğu gibi içerisindeki memberların tanımlanmasını zorunlu kılar!

Neden Interface Kullanırız?



Interface Nasıl Tanımlanır?

- Interface tanımlayabilmek için 'interface' anahtar sözcüğünden faydalanırız.
- Interface'ler referans türlü değişkenlerdir.
- C# programlama dilinde bir Interface'e isim verirken 'I' ile başlaması bir gelenektir.
- Bir class nerede tanımlanabiliyorsa interface te orada tanımlanabilir.

Interface Nasıl Tanımlanır?

```
interface IKullanici
{

}
```

Interface İçerisinde İmzaların Oluşturulması

- Interface içerisinde metot ve property imzaları tanımlanabilmektedir.
- İmzalar tanımlanırken erişim belirteci kullanılmaz, gövdeler tanımlanmaz sadece imzalar tanımlanacaktır.
- Abstract classlarda olduğu gibi imzalarda 'abstract' keywordü kullanılmaz! Çünkü interfaceler sadece sınıflara uygulatılacak imzaları barındırır abstract classlarda olduğu gibi normal memberlara sahip değillerdir!
- Interface içerisinde field tanımı mümkün değildir!

Interface İçerisinde İmzaların Oluşturulması

```
interface IKullanici
{
    void Dogrula();
    int Hesapla();
    int Yas { get; set; }
}
```

Interface Bir Class'a Nasıl Uygulatır

- **Interfacei kullanabilmek için kalıtım sürecinde kullanılan ':' operatörü kullanılır, ancak bu işlemin adı artık kalıtım değil implementasyon olur! Aynı durum abstract class içinde geçerlidir.**
- Interfacelerden yapılan implementasyon sonucu ilgili class interfaceteki imzaları uygulamak zorundadır ancak abstract classlarda olduğu gibi bu işlem için 'override' anahtar sözcüğü kullanılmaz!
- **Interfaceler sınıfların imzasıdır!**

Interface Bir Class'a Nasıl Uygulatır

```
interface IPersonel
{
    void Dogrula();
}

class Personel : IPersonel
{
    public void Dogrula()
    {
    }
}
```

Interfacelerde Çoklu Kalıtım Durumu

- C#'ta bir class sadece tek bir classtan kalıtım alabilmektedir!
- Buna karşın bir class birden fazla Interfacei aynı anda uygulayabilmektedir.
- Ayrıca ilgili classın nesnesi tüm bu interfaceler ile refere edilebilmekte ve soyutlama davranışı gerçekleştirilebilmektedir.

Interfacelerde Çoklu Kalıtım Durumu

```
interface ITopla
{
   int Topla();
}

interface ICarp
{
   int Carp();
}

class Islem : ITopla, ICarp
{
   public int Topla()
   {
      return 5;
   }
   public int Carp()
   {
      return 5;
   }
}
```

Interfaceler Birbirinden Kalıtım Alabilir mi?

- Interfaceler kendi aralarında kalıtım alabilir. Bu işlem implementasyon olarak değil normal kalıtım olarak isimlendirilir. Burada standart kalıtım kuralları geçerli olacaktır. Buradaki en önemli fark Interfaceler kendi aralarında kalıtım uygulasa bile yatayda yine birden çok Interface uygulanabilir!

Interfaceler Birbirinden Kalıtım Alabilir mi?

```
interface ITopla
{
    int Topla();
}

interface ICikar
{
    int Cikar();
}

interface ICarp : ITopla, ICikar
{
    int Carp();
}
```

Hem Kalıtım Hem Implementasyon Uygulanabilir mi?

- C#'ta bir class, farklı bir classtan kalıtım alırken hem de bir yandan interfacei implemente edecekse eğer önce classtan kalıtım almalı, sonrasında da interfaceler tanımlanmalıdır. Buradaki öncelik önemlidir aksi taktirde derleyici hata verecektir.

Hem Kalıtım Hem Implementasyon Uygulanabilir mi?

```
class Canli
{

}

interface ISolunum
{
   void SolunumYap();
}

interface IDolasim
{
   void DolasimYap();
}

class Insan : Canli, ISolunum, IDolasim
{
   public void DolasimYap()
   {
   }
   public void SolunumYap()
   {
   }
}
```